

The goal:

- 👉 understand *why* the stylesheet is written the way it is
- 👉 understand *how* to reuse these ideas in any future HTML/CSS project
- 👉 practice reading CSS from top to bottom with meaning

🌻 STUDENT'S GUIDE TO THE PROJECT STYLE SHEET

(id vs class · background styling · div layout · lists · fonts · cascade · and scroll concepts)

This guide walks you through the components of this stylesheet and the reasoning behind them.

You will learn both **how it is built** and **why it works**.

1. id vs class — How & Why We Use Them

What is an `id`?

- An **id** identifies **ONE unique element** on the page.
- It is used once.
- Example in your project:

```
<div id="page-wrapper">
```

- CSS selects it using:

```
#page-wrapper { ... }
```

- We use it because it represents the **whole main container**, the layout wrapper for all content.

What is a `class`?

- A class identifies **groups of elements that share the same styling**.
- You can apply it many times.
- Example:

```
<div class="card">
```

- CSS selects it with:

```
.card { ... }
```

Why this matters

- `id` = unique structure
- `class` = reusable design

This division keeps your code clean, predictable, and scalable.

2. Background: Properties & Application in the Project

Your stylesheet uses several essential background properties on the `<body>` element :

```
background-color: #e0f2ff47;  
background-image: url("../pics/skies.jpg");  
background-repeat: no-repeat;  
background-position: center center;  
background-attachment: fixed;  
background-size: cover;
```

Meaning of each:

Property	What it does
background-color	fallback color before the image loads
background-image	sets the sky image from your project folder
background-repeat: no-repeat	prevents tiling of the image
background-position: center center	centers image horizontally + vertically
background-attachment: fixed	background stays fixed while content scrolls ("parallax feel")
background-size: cover	image fills all available space of the viewport

Why use it on `<body>`?

Because we want a **full-page sky atmosphere**: a fixed, immersive background that affects the whole experience of the page.

3. Div Layout: How Cards Distribute in Rows & How to Add More

Your `.card` layout uses the classic **float model** (pre-flexbox), like this:

```
.card {  
  width: 30%;  
  margin: 1.66%;  
  float: left;  
}
```

How it works

- **float:left** pushes cards next to each other horizontally.
- Each card takes **30%** of the row.
- Margins make spacing nice and organic.

Why exactly 30%?

Because:

- $30\% \times 3 \text{ cards} = 90\%$
- plus margins → roughly one full row.

This gives **3 cards per row on desktop**.

How to add 3 more cards?

Just copy & paste more:

```
<div class="card"> ... </div>  
<div class="card"> ... </div>  
<div class="card"> ... </div>
```

Float naturally continues the flow:

- Card 4 → moves to the next row
- Card 5 → next to it
- Card 6 → next to that one

What if the screen is narrow?

Your responsive rule handles it:

```
@media (max-width: 800px) {  
  .card {  
    width: 100%;  
    margin: 10px 0;  
    float: none;  
  }  
}
```

That means:

- **Each card = full width**
- Simple vertical stacking
- Looks good on mobile

4. Ordered & Unordered Lists — Why Styled This Way

Ordered Lists (o1)

Your stylesheet resets the list to normal text, then bolds the markers:

```
.card ol > li {  
  font-weight: normal;  
}  
  
.card ol > li::marker {  
  font-weight: bold;  
}
```

Why?

Because it makes the numeric label **more visible** without overwhelming the whole line.

You also have:

```
.card ol .label {  
  font-weight: bold;  
}
```

This bolds only the “short descriptor” like:

- **Habitat:**
- **Peak season:**
- **Latin name:**

This is multi-language–safe and keeps lines clean.

Unordered Lists (`ul`)

Your stylesheet replaces the default bullet with **botanical symbols**:

```
.card ul {  
  list-style: none;  
}  
.card ul > li::before {  
  content: "\2698"; /* flower */  
}
```

Reasoning:

- Default bullets are too generic.
- Flower bullets match the **theme**.
- Different bullet (🌸) for allergy lists communicates context immediately.

This makes nested lists readable and expressive.

5. Fonts & Typography — Imports, Application & Cascading Rules

At the top of your CSS you import two Google Fonts:

```
@import url('https://fonts.googleapis.com/css2?  
family=Borel&display=swap');  
@import url('https://fonts.googleapis.com/css2?  
family=Faculty+Glyphic&display=swap');
```

How fonts cascade

Your `body` sets a system font:

```
font-family: system-ui, ...
```

This is the **base font** for everything on the page.

Then, specific elements override this:

```
#page-wrapper h1 {  
  font-family: "Faculty Glyphic", sans-serif;  
}
```

Cascading rules:

1. **Inline style** → strongest
2. **ID selector rules**
3. **Class selector rules**
4. **Element selectors** (h1, p, etc.)
5. **Inherited defaults** (body → children)

Which one prevails?

The one that is:

- **more specific**, or
- **declared later** in the stylesheet.

How to use this in your favor:

- Want a consistent card title style?
Use `.card h2 { ... }`.
- Want ALL headings to share a visual identity?
Style `#page-wrapper h2, h3 { ... }`.
- Want something global?
Put it in `body`.

This encourages clean hierarchy instead of random overrides.

6. Personal Bonus: Understanding the Scroll Effect

The horizontal scroll (if you add the flex-strip version later) is a modern UI pattern, but even now you already use a **fixed background scroll interaction**:

```
background-attachment: fixed;
```

What this does:

- Content scrolls **independently** from the background.
- Creates a **parallax visual effect**.
- Makes your cards feel like they float above the sky.

If you later replace the float layout with a **horizontal scroll strip**, the concepts are similar:

- The container scrolls horizontally.
- Cards align inside it.
- You give visual cues (“scroll hint”) to indicate overflow.

Scrolling effects reinforce modern usability by:

- Encouraging exploration
- Signalling there is more content
- Keeping the interface visually playful

Final Summary

This stylesheet teaches you the essentials of front-end layout:

Structure

- Use **id** for unique containers
- Use **class** for repeating components

Styling

- Background properties create atmosphere
- Cards use floats and responsive breakpoints

Layout Logic

- 3 cards per row via percentage widths
- Automatic line wrapping
- Mobile stacking via media queries

Typography

- Cascade decides which font or style wins
- Specific selectors override generic ones
- Custom fonts add personality

Lists

- Ordered lists emphasize structure
- Unordered lists communicate meaning (flowers vs allergies)

Scrolling

- Background “fixed” creates depth
- A horizontal strip (if added) enhances navigation